Integrative Project of the 4th Semester

Licenciatura em Engenharia Informática, ISEP - 2025/2026 Porto,

April 2026

Version 2.a

Course coordinators

André Moreira (ASC) - RCOMP

Angelo Martins (AMM) – LAPR4

António Constantino Martins (ACM) - LPROG

Luis Nogueira (LMN) - SCOMP

Nuno Pereira (NAP) – EAPLI

1 Context

In the 2025-2026 academic year, the fourth semester (i.e. 2nd year, 2nd semester) of the Degree in Informatics Engineering (LEI) of the Instituto Superior de Engenharia do Porto (ISEP) adopts a Project-Based Learning (PBL) learning process based on the development of a single project integrating all units of the semester: Applications Engineering (EAPLI), Laboratory and Project IV (LAPR4), Languages and Programming (LPROG), Computer Networks (RCOMP) and Computer Systems (SCOMP). The project, common to all course units, consists of developing the system described in this document in accordance with the general procedures also described in this document.

To develop the project, students must form groups/work teams common to all UC. The objective is to obtain an integrated solution, i.e., a single system encompassing the various components developed.

The operating rules indicated in this document on working methodology and the formation of work teams are guidelines that cover the main existing scenarios. Situations other than those mentioned in this document must be presented to the regent (RUC) of LAPR4, that together with the RUC of the remaining UCs, will decide how to proceed in that specific scenario.

It is important to highlight that the development of project should follow the best practices acquired in the previous semesters, as well as best practices acquired/deepened throughout this semester. The application of these best practices is an integral part of the assessment criteria of the final project.

2 Rules

In the development of the project, it is mandatory to apply the knowledge and skills acquired in all the semester's UC in an integrated and phased manner as presented in Table 1 (the "X" indicates that the UC participates in the respective project sprint).

The development of the project will take place primarily in the PL classes of each UC (including LAPR4), which are dedicated the resolution of problems and applications of techniques related with the contents of each UC. Students must follow the indications of their UC teachers regarding the specificities of each UC in the context of the project. In addition, parts of the project must be developed outside of class time.

	EAPLI	LPROG	RCOMP	SCOMP	LAPR4
Sprint 1	X				X
Sprint 2	X	X		X	X
Sprint 3	X	X	X	X	X

Table 1 – UC participating in each sprint

Deadlines (commit on GitHub before):

- Sprint 1 – 5th April 2026, 20:00
- Sprint 2 – 17th May 2026, 20:00
- Sprint 3 – 14th June 2026, 20:00

2.1 Teams

The project is carried out by teams of 4 students of the same PL class having all the units of the semester, as depicted in Table 2. Each team will operate as an independent development team and the focus should always be on the project as a whole, not in each UC individually.

	EAPLI	LPROG	RCOMP	SCOMP	LAPR4
Student 1	X	X	X	X	X
Student 2	X	X	X	X	X
Student 3	X	X	X	X	X
Student 4	X	X	X	X	X
Total per UC	4	4	4	4	4

Table 2 – Usual scenario for team composition

However, considering the diversity of LEI students enrolled in the 4th semester, other team compositions may be accepted. If the team members aren't enrolled in all courses, the number of team members can go up to 5 students and should be ensured is that there are at least 3 students enrolled in each of the UCs.

Special situations must be presented to the LAPR4 RUC, which together with the RUCs of the remaining UCs, will decide how to proceed in those specific scenarios.

The students should create the team according to the deadlines defined in Moodle and submit it in Moodle. The team is responsible for the accuracy of the information submitted in Moodle.

There will be a repository in Github for each team. These repositories will be created in the context of a Github classroom. The teams will receive information regarding the creation of the repository. Failure to comply with the instructions may negatively affect the team's assessment and the grades. Regarding repository maintenance, as per best practices, Github should be updated on a regular basis as work progresses (e.g. daily updates). Failure to comply – such as doing only 1 or 2 submissions per week, will negatively affect the team's assessment and the grades.

2.2 Work methodology

At the beginning of each sprint, a set of user stories indicating the work to be carried out by the team during the sprint is made available. Thus, the team's responsibility (per sprint):

- analyse this information and, if necessary, request clarifications in a timely manner. Functional questions (which allow, for example, to clarify acceptance criteria) should be asked in sessions with the Product Owner (LAPR4 RUC).
- correctly apply the engineering process and its different activities (e.g., analysis, design, implementation, tests, implantation), leaving the necessary evidence.
- carry out a planning of the tasks to be carried out that considers both the sprint objectives and the existence of possible dependencies between tasks and, whenever necessary, review and change the planning carried out previously.
- carry out a balanced division of tasks (e.g., in terms of effort, knowledge and complexity) among all team members in order to ensure that any element from the team is able to describe and explain how a certain requirement has been implemented/satisfied in the system and also be able to implement requested changes (e.g., in an assessment).

Please note that it is not intended nor acceptable for each team member to dedicate themselves just the realization of tasks that require the application of knowledge of a single UC (e.g., EAPLI or SCOMP). On the contrary, each element must participate in tasks that require the application of knowledge from all the UCs in which he/she is enrolled. Students enrolled in LAPR4 must implement requirements from all UCs of the semester.

Students not enrolled in LAPR4 may not be involved in the implementation of requirements from UCs they are not enrolled in, but they must be able to discuss the whole project.

It is also important to emphasize that it is everyone's responsibility to ensure the smooth running of the team. In the event of conflict, it should first be addressed by the team. If the team fails to solve the problem, help should be asked to the team's Scrum Master (LAPR4 PL teacher).

The team must work regularly (several times a week) and provide evidence of this in GitHub via commits and issues. These will be used in the assessment.

Some courses may have specific requirements about user story assignment.

2.3 Professional and academic conduct

LEI's integrative projects are essential components in the preparation and training of students for their future profession as software engineers. A software engineer assumes very important responsibilities today, where a large part of activities is supported directly or indirectly by software solutions that are designed, implemented and maintained by software engineers. In addition to technical skills, the software engineers must follow one global set of principles that are part of the professional code of ethics.

Students must know and apply principles of ethics and professional conduct. Below are some points taken from the Code of Ethics and Deontology of the Portuguese Engineers Association[1]:

- Engineers are aware of the implications of the integration of technical systems in social, economic and environmental context, which is why, in the development of new technologies, they are concerned with criteria and values such as:
  - the sustainability of the technical systems, throughout their life cycle.
  - fitness for use and safety.
  - the contribution to the health and well-being of the citizens.
  - personal development contributing to common good.
- The fundamental guideline in the creation of new technological solutions will be to maintain open the possibility of free and responsible action, both to current and future generations.
- The engineer must oppose to the fraudulent or harmful use of its work.
- The engineer must seek the best technical solutions, considering the economic and quality constraints.

The practice of the principles of good conduct begins in the academic training process itself. Therefore, principles such as accountability and integrity are of utmost importance. Academic integrity principles published by MIT [2] can be considered general and applicable to our academic environment.

On this subject, it is also important that all students are aware of the Regulamento Disciplinar dos Estudantes do Instituto Politécnico do Porto[3].

2.4 Assessment

Each UC participating in the project will carry out, at the end of each sprint, an individual and/or team assessment (formative and/or summative) of the work carried out.

The criteria used in these assessments are defined exclusively by each UC, and they apply to the technical aspects of the project that fit into the programmatic objectives of each UC. As such, the criteria applied vary and are distinct from one UC to another UC. The results (grades) obtained in a UC do not directly influence the results of the others UC. It's very important that the students apply a suitable work methodology and divide the task taking the assessment in account. Failure to do so may result in negative consequences in the assessment of some UCs.

The assessment of each UC will be in accordance with the respective Curricular Unit Form (FUC) and may include other assessment moments. If these additional assessment moments exist, they will tend to occur before the UC in question begins its participation in the project or after the end of the project development (i.e. during the examination period).

LAPR4 assessment will be focused on the correct application of the scrum methodology and of an iterative and incremental development process. Testing is an important component of this process, so that code coverage of domain and controller packages cannot fall below 90%.

[1] https://www.ordemengenheiros.pt/fotos/editor2/regulamentos/codigo_ed.pdf
[2] http://integrity.mit.edu
[3] https://www.ipp.pt/ensino/informacoes-academicas/regulamentos/Despacho41032013_RegulamentoDisciplinarEstudantesIPP.pdf

3 System specifications

Aircraft flight control is a key component of today's life. There are thousands of flights every day, and air traffic control is responsible for managing them, so that they don't collide mid-air. Unfortunately, many countries have not invested in this area for many years, and many systems are obsolete. That's the case of the USA.

AISafe is a startup that has eyed this market as a potential gold mine and decided to work on a prototype to showcase their ideas to potential buyers. To speed up development, it outsourced the development of parts of the system to several partners. ISEP-DEI Sem4PI was awarded the development of the software to manage part of the back office, namely:

- Basic configuration (register and update aircraft models, aircraft engine models, aircraft and engine manufacturers, air transport companies, airports, etc.)
- Flight management (create, verify, etc.)
- Weather service (AI enhanced weather service to be used by all system instances).

To avoid vendor lock-in, AISafe wants to use a DSL (Domain Specific Language) to describe the flights. The DSL code it's not generated in the system, but in an outside specialized tool. The code is then imported into the system and used for flight simulation.

Finaly, there is also the need to test/simulate flights:

- Simulation of a flight, which means simulation the flight plan to verify the planned path and altitude are valid, as well as checking if the quantity of fuel on board meets the safety regulations. In the beginning, simplified weather information may be used. But the final prototype must use actual weather forecast retrieved from the AI weather service.
- Finally, there is also the need to simulate the flight control of an area. In this one, the simulation involves the coordination of all flights by a flight control service, that may impose small changes to flights to avoid collisions.

AISafe thinks big and aims to become a leader in the flight control business. One important requirement is to support many simultaneous flights. Thus, the simulation of many flights may not be possible using just one machine. The system should be scalable, so that the simulation can be parallelized. A possibility is to divide the air space in several subareas and use a flight control service for each subarea. Of course, a flight may cover several subareas.

3.1 Use cases

The **main use cases**[4] agreed with the client are depicted in Figure 1. Six main actors were identified:

- Admin
- Backoffice Operator
- Air transport company collaborator
- Pilot
- Flight Control Operator
- Weather Person

[4] Other use cases might be necessary and added throughout the project implementation. Missing use cases that the team deems necessary must first be validated with the customer.

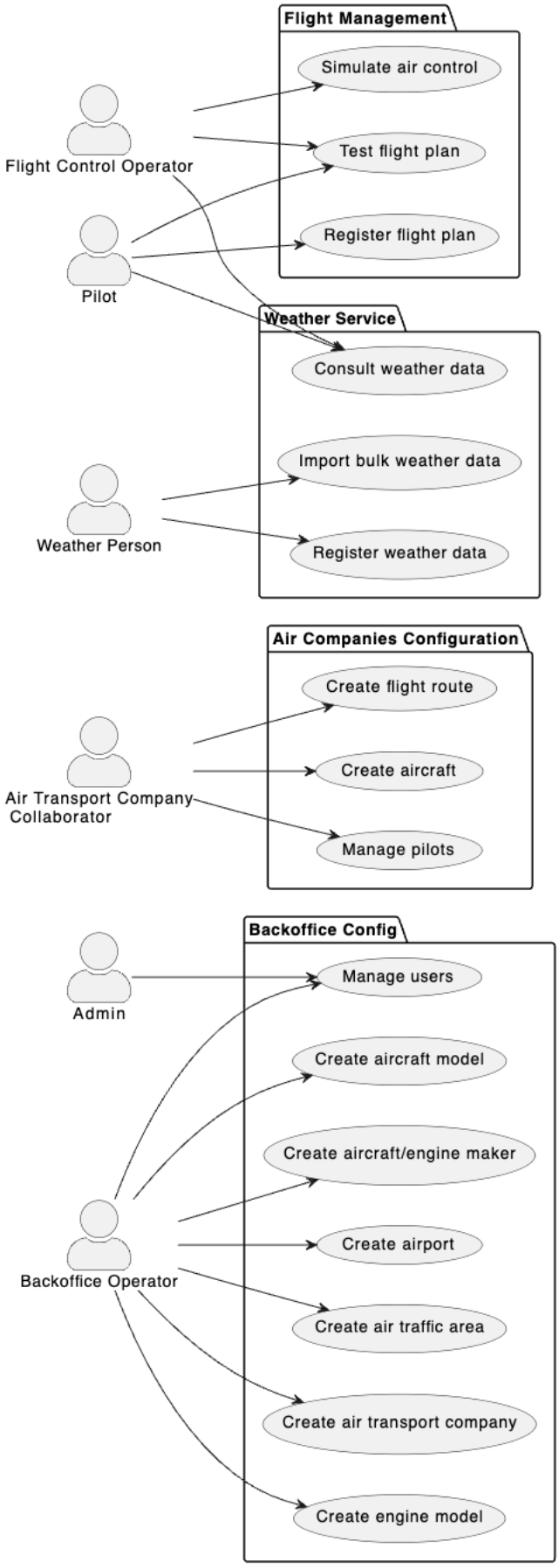


Figure 1 Main use cases

3.1.1 Users and roles

A user is someone with access to the system. It is identified by a unique valid email from the list of valid email domains. A user also has a name and phone number. Users must authenticate into the system to do anything.

An Alsafe user needs to have an active security clearance that automatically expires at a given date. They also need to have periodic (per regulations 5 years) skills assessment. The Admin or the Backoffice operator can update this information.

At the moment, AISafe's management hasn't yet decide if a user can have multiple roles in the system. Right now, none has, but that may happen it the future. The number of roles is also poised to rise in the future.

3.1.2 Aircraft and engine makers

These are not direct clients of the AISafe's system, thus only basic information (name and country?) needs to be stored in the system. Please take in account that country names a prone to changes, albeit other information (e.g. ITU code) usually doesn't change a lot.

3.1.3 Air transport companies

AISafe aims to offer its system globally. Air transport companies are not the direct client, but they use the system to register aircraft and flights. Backoffice Operators can add air transport companies' collaborators (ATCC) of the companies registered in the system. These will be able to register aircraft, flight routes and manage pilots.

An Air Transport Company may have several collaborators (ATCC and pilots) interacting with the system; thus, a list of these collaborators must be stored for each company. Email (valid company email), name and position will probably be enough for the purpose of the system. The set of active collaborators will change over time.

3.1.4 Flight control

AISafe aims to dominate the flight control market globally with its new system. Its main clients are flight control entities. They usually aren't private companies (e.g. FAA and Eurocontrol), but they can be. Security is a main concern, so the collaborators of these entities with access to the system (Flight Control Operators – FCO) are directly managed by the system administrator. They follow the same rules that apply to regular users.

Base system information (aircraft models, airports, flight control areas, etc.) is fed into the system by Backoffice Operators, so that the clients are not overloaded. FCOs only must test flight plans and simulate air control.

The objective of testing a flight is to verify the aircraft carries enough fuel and meets the minimum altitude requirements. The minimum quantity of fuel is the quantity of fuel in the aircraft at the end of the flight required by the air control area of the flight destination airport.

Air control simulation combines the simulation of all flights in the area. If there are flights coming from other areas, they are included the moment they enter the area. This simulation it is important to detect possible air safety violations (e.g. two flights in close proximity).

The system must provide the tools to test/simulate the flights.

3.2 Definitions

The simulation of flights will be based on the applicable classical mechanics (“Classical mechanics”, 2015) and fluids laws, especially the effect of air drag and lift. Some simplifications on aircraft and flight models must be made for the analysis to be attainable using a PC. The simulation models actual flight instances and their execution, abstracting away from both route planning and operational details such as crew assignments.

The main concepts involved are:

- Flight: Connection between two airports. There may be direct flights and flights with one or more stops. Only direct flights will be considered. A flight has a unique flight designator, which is the concatenation of the airline designator, xx, and the numeric flight number, n(n)(n)(n), plus an optional one-letter "operational suffix" (a). Therefore, the full format of a flight designator is xxn(n)(n)(n)(a).
  - Type (regular or charter)
  - Departure day (or days of the week for regular flights and actual date for a charter) and time.
  - Scheduled arrival (local time).
- Segments: A linear path connecting two points. A segment is characterized by:
  - Coordinates of the beginning node
  - Coordinates of the ending node
  - Allowed altitudes slots
  - Width for each altitude slot
  - Wind direction (angle) relative to North and speed (m/s)
- Node/Junction: connection between two or more segments, so that the aircraft may move between segments.
- Airport: Is characterized by:
  - Name
  - Town
  - Country
  - IATA airport code
  - ICAO airport code
  - Coordinates
  - Altitude
- Aircraft model: motorized commercial aircraft. There are many types of motorizations: turboprop, turbofan, turbojet, ram jet, electric propeller, etc. An aircraft model is characterized by:
  - Model ID
  - Maker
  - Type (passenger, cargo, mixed)
  - Motorization
  - Empty weight
  - Maximum take-off weight (MTOW)
  - Maximum Zero Fuel Weight (MZFW)
  - Maximum fuel capacity
  - Service ceiling
  - Cruise speed
  - Wing area
  - Drag coefficient (Cd)
  - Lift coefficient (Cl)
- Aircraft: aircraft of a specific model. It is characterized by:
  - Registration (unique ID)

- Company
- Cabin configuration (number of seats in each class)
- Number of elements of the flight crew

3.3 Aircraft Physics

The lift force applied to the aircraft is given by:

$$L = \mathrm{Cl}\frac{\rho v^2}{2} A$$

L = lift force acting on the aircraft (N)

Cl = Lift coefficient

A = area of the wings (m^2)

ρ = density of air (kgm^{-3})

v = velocity of aircraft relative to the air (ms^{-1})

The drag force applied to the aircraft is given by:

$$D = \mathrm{Cd}\frac{\rho v^2}{2} A$$

D = drag force acting on the aircraft (N)

Cd = drag coefficient

A = area of the wings (m^2)

ρ = density of air (kgm^{-3})

v = velocity of aircraft relative to the air (ms^{-1})

Thus, we have the lift coefficient

$$C_L = \frac{2mg}{\rho A v^2}$$

And the drag coefficient

$$C_D = C_{D0} + \frac{{C_L}^2}{\pi ARe}$$

Where AR is the aspect ratio (wing $span^2$/wing area).

Thrust is a parameter supplied by the manufacturer, usually at sea level and at two speeds: stopped and at cruise or maximum speed. One can assume that thrust will follow a linear behaviour between these speeds. Thrust decreases with altitude according to the following expression:

$$T_{altitude} = T_{Sea\ level}\left(\frac{\rho}{\rho_o}\right)^{.90}$$

Precise fuel consumption estimation is out of the scope of this project. It is an extremely complex process and there is little publicly available data for real aircraft. Thrust specific fuel consumption (TSFC) describe the fuel efficiency of an engine design with respect to thrust output, i.e. fuel consumption (grams/second) per unit of thrust (kN) (SI units). But it can be expressed in other units, such us N/N/S (SI) our in US units.

The range of an aircraft in cruise flight cam be estimated by:

$$R = \frac{TAS}{TSFC}\left(\frac{L}{D}\right) ln\left(\frac{W_i}{W_f}\right)$$

TAS = True Air Speed (relative to air)

TSFC = Thrust specific fuel consumption (N/N/S)

L = Lift

D = Drag

W_i = Initial weight

W_f = Final weight

Please note that the lift/drag relation is not constant during flight. The references include links to further information on engine thrust and fuel consumption. They are a good starting point for research.

The density of jet fuel JET A-1, the one most widely used around the world, at 15 °C is 0.804 kg/l (6.71 lb/US gal). The specific energy is 42.80 MJ/kg.

3.4 Flight plans

A flight must have a valid flight plan to take place. A flight takes place in the context of a route, so that one could say that a flight is a “instantiation” of a route, including flight specific information like the date/time, aircraft and load (people and cargo). The flight plan describes how the flight takes place, including the fuel load.

To avoid vendor lock-in, AISafe requires that flight plans are described using a Domain Specific Language (DSL). This DSL must be imported, validated and processed by the system, serving as a formal input for flight plan creation and validation.

The specification, validation and processing of DSL are core responsibilities of the Languages and Programming (LPROG) course and must be implemented using lexical analysis, syntactic analysis and semantic validation.

3.4.1 Core Flight DSL (Conceptual Model)

A flight plan represents a complete flight operation and may include multiple take-offs and landings. A flight is composed of one or more flight legs, where each leg is a single, non-stop journey segment between two airports, defined by one take-off and one landing.

- A flight aggregates global information, such as its identifier and type.
- Each leg contains its own operational data, including departure, arrival, segments and fuel.

This structure allows the representation of both direct flights and flights with intermediate stops, while keeping the DSL expressive and aligned with the problem domain.

3.4.2 Informal Lexical Specification

The Flight DSL is a textual language composed of the following lexical elements:

- Keywords: Reserved words defining the structure of the language (e.g., flight, leg, departure, arrival, route, segment, fuel).
- Identifiers: Used to represent flight identifiers and named entities.
- Airport codes: Represented using IATA (3-letter) or ICAO (4-letter) codes.

- Numeric literal: Integers and floating-point numbers, used for coordinates, altitudes, widths, wind speed and fuel quantities.
- Date and time literals: Used to represent departure dates and times.
- Units: Used to qualify numeric values (e.g., kg, l, m, m/s).
- Separators and symbols: Such as { } ( ) [ ] , : ;.
- Whitespace and line breaks: May be freely used to improve readability and have no semantic meaning.

Keywords are case-insensitive. Identifiers and literals are case-sensitive. Other lexical elements may be introduced, if they are formally specified, justified, and compatible with the Core Flight DSL

3.4.3 Informal Syntax Specification

General structure

A flight plan is defined by a flight block, which contains:

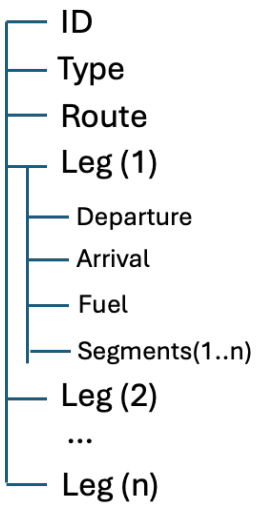
- A unique flight identifier;
- A flight type;
- One or more leg blocks.

Each leg represents a complete flight phase and must include:

- Departure information;
- Arrival information;
- A route definition;
- One or more flight segments;
- Fuel information.

The DSL is hierarchical and block-based.

Informal structure



3.4.4 Formal Processing Requirements (ANTLR)

The Core Flight DSL must be processed using ANTLR:

1. Define a formal grammar, including:
   - Lexer rules;
   - Parser rules.
2. Perform lexical and syntactic validation of flight plan files.

3. Implement parse tree traversal using listeners or visitors.
4. At least one visitor and one listener must be used.
5. Build an internal representation of the flight plan (AST or domain objects).
6. Produce clear and informative error messages, including:
   - Error type;
   - Line and column information.
7. When necessary, the implementation language to be used must be Java.

3.4.5 Semantic Validation Rules

After successful syntactic validation, the system must perform semantic validation to ensure coherence with the domain.

Flight-level rules:

- The flight identifier must be unique within the file.
- The flight type must be one of the allowed values.

Leg-level rules:

- Each leg must contain exactly:
  - One departure;
  - One arrival.
- Each leg must contain at least one segment.
- Fuel quantity must be:
  - Strictly positive;
  - Associated with a specific leg.

Leg sequence coherence:

- The arrival airport of leg n must match the departure airport of leg n+1.
- The arrival time of leg n must precede the departure time of leg n+1.

Route coherence:

- Route origin must match the first leg's departure airport.
- Route destination must match the last leg's arrival airport.
- In a flight, the same airport cannot be visited twice.

Segment coherence:

- Segment start and end coordinates must be different.
- Segments must belong to exactly one leg.

Domain consistency:

- Airport codes must respect the defined format.
- Numeric values (altitudes, widths, wind speed and fuel) must be positive.
- Date and time values must represent valid calendar values.

All semantic errors must be collected and reported in a single execution.

3.4.6 Explicitly Out of Scope

Semantic validation and others do not include:

- Flight physics calculations.
- Fuel consumption estimation.

- Safety or collision detection.
- Weather impact computation.
- Flight simulation.
- The method of integration of the ANTLR solution in the project.

These aspects are addressed by other system components.

3.4.7 Language Extension and Innovation

Although a Core Flight DSL is specified in this document, teams are strongly encouraged to extend the language beyond the minimal requirements.

Extensions may include:

- New lexical elements (additional keywords, literals, units, etc.);
- Additional syntactic constructs;
- Additional semantic validation rules aligned with the problem domain.

Any extension must:

- Preserve compatibility with the Core Flight DSL;
- Be formally specified in the grammar;
- Be properly documented and justified;
- Be fully supported by lexical, syntactic and semantic validation.

Innovative, coherent and well-designed extensions, particularly those that improve expressiveness, clarity or domain alignment, will be positively valued in the assessment.

4 User stories

4.1 Product Backlog

US001 Technical constraints

As Project Manager, I want the team to follow the technical constraints and concerns of the project.

These constraints and concerns are described in section 5.

US002 Project repository

As Project Manager, I want the team to use the defined project repository in GitHub and setup a GitHub tool for project management.

US003 Project structure

As Project Manager, I want the team to configure the project structure to facilitate/accelerate the development of upcoming user stories.

Define the structure of the project to support the envisioned architecture, including support for adopted technologies (e.g., ANTLR)

US004 Continuous integration server

As Project Manager, I want the team to setup a continuous integration server.

GitHub Actions/Workflows should be used

US005 Automated deployment

As Project Manager, I want the team to add to the project the necessary scripts, so that build/executions/deployments can be executed effortlessly in a Unix compatible machine.

Include scripts for all the major tasks and execution of applications

US010 Domain model

As Project Manager, I want the team to elaborate a Domain Model using DDD. Because AISafe domain is complex, involving distinct areas and complex business rules. We will adopt DDD as a framework for tackling complexity in our software solution, and ensure the software is a valid implementation of the business needs.

US011 Aggregate justification

As Project Manager, I want the team to illustrate one representative scenario per aggregate with a sequence diagram and a short explanation, so that the aggregate's responsibility is demonstrated through an invariant it enforces. This justification will make aggregate design decisions more explicit and easier to review by the project manager.

US030 Authentication and authorization [5]

As a Project Manager, I want the system to support and apply authentication and authorization for all its users and functionalities.

US031 Register users

As Administrator, I want to be able to register users of the backoffice.

[5] You may, and are advise to, use existing frameworks to handle these authentication and user management aspects. They are a necessity but are not the main business focus of the system. The team should work primarily on use cases that have meaningful business value.

This must also be achieved by a bootstrap process.

US032 Disable/enable users

As Administrator, I want to be able to disable/enable users of the backoffice.

US033 List users

As Administrator, I want to be able to list the users of the backoffice, including their status.

US041 Register weather data

As a Weather Person, I want to register weather data in the system for a specific air control area.

US042 Import bulk weather data

As a Weather Person, I want to import bulk weather data into the system.

Weather data might be from multiple external weather service providers, so the system can aggregate data from various sources for better accuracy. While a simple CSV file format will be available for initial system development, the system should be easy to expand to new weather data sources.

US043 Consult weather data

As a Weather Person, a Pilot, or a Flight Control operator, I want to consult weather data in the system in a given day and in a specific air control area.

US44 Weather Person remote access

As a Weather Person, I want to remotely access to the system in order to upload weather data.

Acceptance criteria:

- A specific TCP-based network client application is required to communicate with the server application embedded in the system.
- The client application interaction with the system must be limited to the TCP connection; remarkably, any direct interaction with the database is unacceptable.
- All the Weather Person user stories must be remotely available by using this client application.
- Authentication and authorization must be enforced.

US050 Register an air control area

As a Backoffice Operator, I want to register an air control area. The area code must be unique in the system. Geographic boundaries must be valid.

This must also be achievable by a bootstrap process.

US052 Create an airport

As a Backoffice Operator, I want to register an airport in a given air control area. An airport must be associated with exactly one air control area. The airport has an ICAO and IATA code that must be unique worldwide. An airport has location coordinates that must be valid and an elevation in meters above sea level.

This must also be achieved by a bootstrap process.

US055 Create an aircraft model

As a Backoffice Operator, I want to register a new aircraft model. Any model must have at least one engine model (let's assume that all engines are of the same type). Aircraft has a model name and

manufacturer, and their combination must be unique. At least one certified engine model must be associated. An aircraft model will have a type, maximum range and other flight characteristics.

This must also be achieved by a bootstrap process.

US056 Create an aircraft engine model

As a Backoffice Operator, I want to register a new engine model to be used by aircrafts. The model's name and manufacturer combination must be unique. Other important information regarding the type (see section 3.2), power, fuel, and efficiency should be included.

This must also be achievable by a bootstrap process.

US057 Add an engine model to an aircraft model

As a Backoffice Operator, I want to add an engine model to an aircraft model's list of certified engines. Engine type must be compatible. The same engine model cannot be added twice to the same aircraft. An aircraft model might have several aircraft variants (combinations of model and engine configuration)

US058 Remove an engine model from an aircraft model

As a Backoffice Operator, I want to remove an engine model from an aircraft model's list of certified engines. This is not possible if there are actual aircrafts using that model. Aircraft model must retain at least one certified engine model.

US060 Register an air transport company

As a Backoffice Operator, I want to register an air transport company. A company has a company name, and IATA (2 letters) and ICAO code (2-3 letters) that must be unique.

This must also be achievable by a bootstrap process.

US061 Add a costumer's collaborator

As a Backoffice Operator, I want to register a costumer's collaborator.

A costumer may be an air transport company or an air control area. The collaborator will also be a user of the system. Thus, each collaborator will be a distinct system user. There is no need to verify that the collaborator's email is in the customer's domain. This could be extremely difficult or even impossible, and many users have emails outside the company's main domain.

This must also be achievable by a bootstrap process.

US062 List costumer's collaborators

As a Backoffice Operator, I want to list all collaborators of a given customer. It should not list disabled collaborators.

US063 Edit a customer's collaborator

As a Backoffice Operator, I want to edit the information (email and phone number) of a customer's collaborator. All other information cannot be changed.

US064 Disable a customer's collaborator

As a Backoffice Operator, I want to disable a customer's collaborator so that he may not use the system anymore.

US070 Add an aircraft to an air transport company

As an Air Transport Company Collaborator, I want to add an aircraft to my company's fleet.

The aircraft is a of a given model, including the engines, and the number of seats of each type/class must be provided. Total number of seats cannot exceed the model's capacity. An aircraft is identified

by the aircraft registration number, that is unique. Also, an aircraft is registered in a country (it may not be the company's home country). An aircraft also has an operational status.

US071 Decommission an aircraft

As an Air Transport Company Collaborator, I want to retire an aircraft from my company's fleet. It's not possible to create flights for a retired aircraft, nor it is possible to retire an aircraft that has pending flights registered in the system.

US072 List an air transport company's fleet

As an Air Transport Company Collaborator, I want to list my company's aircraft fleet.

US072a List an air transport company's fleet by model

As an Air Transport Company Collaborator, I want to list my company's aircraft of a given model.

US072b List an air transport company's fleet by maker

As an Air Transport Company Collaborator, I want to list my company's aircraft of a given maker.

US072c List an air transport company's fleet by capacity

As an Air Transport Company Collaborator, I want to list my company's aircraft of a given capacity (number of passengers).

US072d List an air transport company's fleet by age

As an Air Transport Company Collaborator, I want to list my company's aircraft of a given age.

US073 Create a flight route

As an Air Transport Company Collaborator, I want to add a flight route for my company. A route is between two airports (start and end) and has a name with 2 letters (my company's initials) and up to 4 numbers (e.g. TP123). The route's name must be unique.

US074 Delete a flight route

As an Air Transport Company Collaborator, I want to deactivate a flight route from a given date onwards.

No more flights can be created on a deleted flight route. But a route cannot be deleted if there are planned flights after that date.

US075 Add a pilot

As an Air Transport Company Collaborator, I want to add a pilot to my company.

A pilot is a system's user. A pilot is certified to pilot one or more aircraft models.

US076 List pilot roster

As an Air Transport Company Collaborator, I want to list my company's pilot roster.

US077 Remove a pilot

As an Air Transport Company Collaborator, I want to make a pilot inactive in my company's roster. Cannot deactivate a pilot with flight plans assigned.

US78 Air Transport Company Collaborator remote access

As an Air Transport Company Collaborator, I want to remotely access the system using the Air Transport Company App.

Acceptance criteria:

- A specific TCP-based network client application is required to communicate with the server application embedded in the system.
- The client application interaction with the system must be limited to the TCP connection; remarkably, any direct interaction with the database is unacceptable.
- All the Air Transport Company Collaborator user stories must be remotely available by using this client application.
- Authentication and authorization must be enforced.

US080 Create a flight plan

As Pilot, I want to register a flight plan for a route. I must add the aircraft, departure date/time, fuel quantity, pilot. The pilot must be of the route's company. Flight plan status is set to "draft" when created and must undergo a multi-step validation process.

US081 Create a flight plan from a file

As a pilot, I want to create a flight plan from a file.

There may be multiple file formats, and I want to validate the file format before using it.

Acceptance criteria:

- The flight plan file must conform to the Core Flight DSL.
- The file must be validated through lexical, syntactic and semantic analysis.
- Invalid files must produce meaningful error messages.
- Only valid flight plans may be imported and used by the system.

US082 Insert weather data in a flight

As a Pilot, I want to add to flight plan of mine the weather data. If the flight plan has been previously tested, the test is deemed void because of the new weather data.

US083 Flight DSL specification and validation

As a Project Manager, I want the team to specify and implement the Flight Description DSL, so that flight plans can be formally defined and validated.

Acceptance criteria:

- The informal lexical and syntactic specification of the DSL is documented.
- A formal grammar is defined using ANTLR.
- The system performs lexical and syntactic validation.
- An internal representation (AST or domain objects) is produced.
- Invalid inputs generate clear and informative error messages.

US085 Test/validate flight plan

As Pilot, I want to test/validate a flight I've made.

Acceptance criteria:

- The validation must include validation of the flight plan described using the DSL and its test.
- The test component must be implemented in the C language.

US86 Pilot remote access

As a Pilot, I want to remotely access to the system using the Air Transport Company App.

Acceptance criteria:

- A specific TCP-based network client application is required to communicate with the server application embedded in the system.
- The client application interaction with the system must be limited to the TCP connection; remarkably, any direct interaction with the database is unacceptable.
- All the Pilot user stories must be remotely available by using this client application.
- Authentication and authorization must be enforced.

US 100 Simulate flights in a given area

As a Flight Control Operator, I want to simulate flights in area. Simulations have many parameters, such as time range, geographic area, included flights, weather conditions, safety thresholds, performance settings. All required parameters should be validated.

Acceptance criteria:

- This component must be implemented in C and must utilize processes, pipes, and signals.
- The system should fork a new process for each flight
- Each flight process should execute its designated flight plan.
- Pipes should facilitate communication between the main process and each flight process.
- The main process should track aircraft positions over time using an appropriate data structure.

US101 Capture and process flight movements

As a simulation process, I want to receive movement commands from flight processes so that I can track aircraft positions over time.

Acceptance criteria:

- Each flight process must send position updates to the main process via a pipe.
- The main process should track aircraft positions.
- The system must store past positions to anticipate and detect potential safety violations.

US102 Detect aircraft safety violations in real time

As a simulation system, I want to continuously monitor aircraft positions for overlaps so that I can identify and report safety violations.

Acceptance criteria:

- The system must detect when two or more aircraft may eventually violate safety rules.
- Upon detecting a violation, the system should log the event and notify the involved aircraft via signals.
- Each flight process must handle the received signal and notify the system user with a message.
- When a flight process receives a SIGUSR1 (violation detected), it should block other signals while handling it.
- The system should allow early termination if safety violations exceed a predefined threshold by sending termination signals to aircrafts.
- Flight processes properly handle termination signals and perform any necessary cleanup.

US103 Synchronize flight execution with a time step

As a simulation engine, I want to synchronize aircraft movements based on time steps so that I can accurately simulate real-world execution.

Acceptance criteria:

- The simulation must progress step by step.
- Each flight process should send position updates at defined intervals.

- The main process must ensure all updates for a given time step are processed before advancing to the next step.

US105 Initialize hybrid simulation environment with shared memory

As a Flight Control Operator, I want to start the simulation with a multi-threaded parent process and multiple child flight processes communicating through a shared memory area, so that the system efficiently coordinates simulation data across processes.

Acceptance criteria:

- The parent process spawns dedicated threads for its functionalities.
- Each flight is launched as an independent process.
- A shared memory segment is allocated and properly initialized for inter-process communication.
- Flight processes are configured to use semaphores for synchronization.
- This component must be implemented in C and must utilize threads, mutexes, condition variables, and signals.

US106 Implement function-specific threads in the parent process

As a PO, I want the simulation controller parent process to have at least two dedicated threads (one for safety violation detection and one for report generation), so that each functionality operates concurrently and independently.

Acceptance criteria:

- The parent process creates a safety violation detection thread responsible for scanning the shared memory for aircraft flight conflicts.
- A report generation thread is created to compile simulation results and respond to safety violation events.
- Any additional thread that you deem appropriate for any of the required functionalities.
- Threads are managed using mutexes and condition variables for internal synchronization.

US107 Notify report thread via condition variables upon safety violation detection

As a PO, I want the simulation system safety violation detection thread to notify the report generation thread through condition variables when a safety violation occurs, so that the report is updated in real time with accurate information.

Acceptance criteria:

- The safety violation detection thread monitors the shared memory for flight conflicts.
- Upon detecting a safety violation, the thread signals the report generation thread using condition variables.
- The report generation thread, waiting on the condition variable, immediately processes the safety violation event and logs it.
- Proper mutex locking is used to ensure thread-safe notification.

US108 Enforce step-by-step simulation synchronization

As a PO, I want the simulation engine to synchronize the simulation's step-by-step progression using semaphores, so that all flight processes and parent threads advance in lockstep through each simulation time step.

Acceptance criteria:

- Semaphores are used to control the progression of each simulation time step.

US109 Generate and store final simulation report

As a Flight Control Operator, I want a comprehensive report that details the simulation outcomes — including flight execution statuses, safety violation events (with timestamps, positions and velocity vectors), and overall validation results —, so that I can assess the safety and performance of the flights post-simulation.

Acceptance criteria:

- The report generation thread aggregates data once the simulation concludes.
- The report includes the total number of flights, individual execution statuses, and detailed safety violation events.
- The final validation result (pass/fail) is clearly indicated.
- The complete report is saved to a file for future reference.

US110 Integrate environmental influences into simulation

As a PO, I want the simulation to incorporate environmental factors such as wind into the simulation, so that the flight paths become more realistic and adapt to dynamic conditions.

Acceptance criteria:

- The parent process spawns an additional “environment” thread at simulation start
- This thread loads environmental configuration (wind speed/direction) from a weather service
- Environment data is written into the shared memory segment at each time step

US111 Generate a simulation report

As a Flight Control Operator, I want to receive a summary of the simulation results so that I can determine if the programmed flights are safe to run.

Acceptance criteria:

- The system must generate a report and store it in a file.
- The report should include the total number of flights and their execution status.
- If safety violations occur, the report must list timestamps and positions.
- The report should indicate whether the scheduled flights plan passed or failed validation.

US112 Monthly Report Generation

As a Flight Control Operator, I want to generate a monthly statistics report.

This will be one of a number (e.g. compliance, incident, etc) of reports the system will have to generate in the future and this work should be foundational to these to guarantee all follow a consistent branding and structure. We imagine each report type has a specific way of collecting data, specific sections and graphics.

US113 External logging of flights progress

As a Flight Control Operator, I want the remote logging of simulated flights.

Acceptance criteria:

- A specific UDP based network server application, the Flights Logging Server, receives updates from flight processes.
- On each simulation step, every running flight process must send an update (UDP datagram) to the Flights Logging Server.
- The Flights Logging Server must store the received data in a separate file for each flight.

US114 Simulated flights visualization

As a Flight Control Operator, I want to view the progress of the simulated flights.

Acceptance criteria:

- The Flights Logging Server application contains an HTTP server to provide a status page to clients running a standard web browser.
- The data about the running simulated flights may be presented as a table, but it must be automatically updated to show the step-by-step progress of the simulation.
- AJAX must be used to update the presented data without the need of reloading the page.

5 Non-Functional Requirements (NFR)

This section presents some specific non-functional requirements. This includes some constraints and concerns that should be considered when designing and implementing the solution.

NFR01 - Project management using scrum

Scrum should be used for project management. The group's LAPR4 PL teacher will be the Scrum Master and there will be one weekly scrum meeting with the scrum Master during the LAPR4 PL class.

The Development Process document, available at Moodle, provides detailed information regarding this requirement.

NFR02 - Technical Documentation

Project documentation should be always available on the project repository ("docs" folder, markdown format) and, when applicable, in accordance with the UML notation. The development process of every US (e.g.: analysis, design, testing, etc.) must be reported (as part of the documentation). Whenever possible, the PlantUML tool shall be used to generate diagrams. The diagrams' source files and the actual diagrams in a vector format (PNG) must be included in the repository.

NFR04 - Source Control

The source code of the solution as well as all the documentation and related artifacts should be versioned in a GitHub repository to be provided to the students. Only the main (master/main) branch will be used (e.g., as a source for releases).

NFR05 - Continuous Integration

The Github repository will provide night builds with publishing of results and metrics. The project will use Maven as build automation tool.

NFR06 – Commits in a continuous integration environment

Any commit must change the system from a valid state to another valid state. If a commit leads to the system failing to compile and pass the tests, the author will get a grade of zero in LAPR4 in that sprint.

NFR03 – Unit testing requirements

It is recommended that each team member adopts a test-driven development approach locally. Test coverage of Java controller and domain packages cannot be below 90% at any time.

Unit testing should follow industry best practices, such as the AAA convention.

NFR07 - Deployment and Scripts

The repository should include the necessary scripts to build and deploy the solution in a Linux system. It should also include a readme.md file in the root folder explaining how to build, deploy and execute the solution.

NFR08 - Database by configuration

The system must support that data persistence is done either "in memory" or in a relational database (RDBMS). Although in-memory database solutions should be used during development and testing, the solution must include a final deployment where a remote persistent relational database is used. The system should have the ability to initialize some default data.

NFR09 - Authentication and Authorization

The system must support and apply authentication and authorization for all its users and functionalities.

NFR10 - Programming language

The solution should be implemented using Java as the main language. Other languages can be used in accordance with more specific requirements.

NFR11 – LPROG Assessment

The quality of:

- the DSL conceptual design.
- grammar clarity and correctness.
- robustness of lexical and syntactic analysis.
- completeness and rigor of semantic validation.
- quality of error reporting.
- innovation and justified extension of the DSL (lexical and syntactic levels).
- internal representation design (AST/domain model).

Will be central elements in the assessment of the Languages and Programming course unit.

Projects that go beyond the minimal Core DSL specification in a coherent and well-structured manner will be positively valued.

6 US Planning

6.1 Sprint 2: EAPLI-Related User Stories

The following table presents the EAPLI-related User Stories required for Sprint 2. Each group is responsible for distributing mandatory USs among group members. Extra/optional USs implemented will also be considered when reviewing the work. **Team size is the number of EAPLI students in the group**.

For example, a group with 3 students enrolled in EAPLI will have to implement the user stories in the column "3 people". US030 is common to all students and each student will have to implement a set of other 3 user stories from that column.

		Team Size (Number of EAPLI students)			
		2 people	3 people	4 people	5 people
US030	Authentication and authorization	M	M	M	M
US031	Register users	M	M	M	M
US032	Disable/enable users	E	E	E	M
US033	List users	E	E	E	M
US050	Register an air control area	M	M	M	M
US052	Create an airport	M	M	M	M
US055	Create an aircraft model	M	M	M	M
US056	Create an aircraft engine model	M	M	M	M
US057	Add engine model to aircraft model	E	E	E	M
US058	Remove engine model from aircraft model	E	E	E	E
US060	Register an air transport company	M	M	M	M
US061	Add a customer's collaborator	E	M	M	M
US062	List customer's collaborators	E	M	M	M
US063	Edit customer's collaborator	E	E	E	E
US064	Disable customer's collaborator	E	E	E	E
US070	Add aircraft to air transport company	E	M	M	M
US071	Decommission aircraft	E	E	M	M
US072	List company fleet	E	E	M	M
US041	Register weather data	E	E	M	M

How to read this table: find the column matching your team's total size. M = mandatory US for that team size, E = extra/optional.

M	Mandatory
E	Extra

6.2 LPROG related user stories

US081 and US083 will have to be implemented. All students enrolled in LPROG must be involved in the implementation of these user stories. Please discuss with your LPROG PL teacher the work distribution.

6.3 SCOMP related user stories

US100, US101, US102 and US103 will have to be implemented. All students enrolled in SCOMP must be involved in the implementation of these user stories. Please discuss with your SCOMP PL teacher the work distribution.

7 References and Bibliography

Classical mechanics. (2026). In Wikipedia, The Free Encyclopedia. Retrieved February 2, 2026, from https://en.wikipedia.org/wiki/Classical_mechanics

Drag Coefficient (2026). NASA Glenn Research Center. Retrieved February 2, 2026, from https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/drag-coefficient/

Turbofan Thrust (2026). NASA Glenn Research Center. Retrieved February 2, 2026, from https://www.grc.nasa.gov/www/k-12/airplane/turbfan.html

Turbofprop Thrust (2026). NASA Glenn Research Center. Retrieved February 2, 2026, from https://www.grc.nasa.gov/www/k-12/airplane/turbprp.html

Turbojet Thrust (2026). NASA Glenn Research Center. Retrieved February 2, 2026, from https://www.grc.nasa.gov/www/k-12/airplane/turbth.html

Thrust specific fuel consumption. (2026). In Wikipedia, The Free Encyclopedia. Retrieved February 2, 2026, from https://en.wikipedia.org/wiki/Thrust-specific_fuel_consumption

Energy efficiency in transportation. (2026). In Wikipedia, The Free Encyclopedia. Retrieved February 2, 2026, from https://en.wikipedia.org/wiki/Energy_efficiency_in_transport

Specific impulse. (2016, November 27). In Wikipedia, The Free Encyclopedia. Retrieved February 2, 2026, from https://en.wikipedia.org/wiki/Specific_impulse

Jet fuel. (2026). In Wikipedia, The Free Encyclopedia. Retrieved February 2, 2026, from https://en.wikipedia.org/wiki/Jet_fuel

https://www.world-airport-codes.com

https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/foilsimstudent/

https://www.grc.nasa.gov/www/k-12/UndergradProgs/index.htm

8 List of versions

V1	Initial version